Title: Chess and Graph Theory – Modeling Strategy with Minimax and Game Trees

Course: INFR-1010U – Discrete Mathematics

Student: Muhamad Taaha Mahmood

ID: 100961329

Date: July 26, 2025

Table of Contents

1. Introduction 1

2. Mathematical Concepts 2

   2.1 Graph Theory in Chess

   2.2 Game Trees and Decision Making

   2.3 Search Algorithms: DFS, BFS

   2.4 Minimax and Alpha-Beta Pruning

   2.5 Knight's Tour Problem

3. Application: AI Chess Engine 5

   3.1 Move Evaluation and Heuristics

   3.2 Implementing Graph-Based Search

   3.3 Visual Demonstrations

4. Conclusion 7

5. References 8

1. Introduction

The principles of discrete mathematics serve as the foundation for many software related principles including algorithms and artificial intelligence. Chess engine programs that simulate intelligent chess depends on discrete mathematics for their development. The chess engines depend on graph theory concepts including nodes and edges together with trees to evaluate millions of possible moves and select the best strategy.

A graph model shows the chessboard through vertices which are squares and edges which represent legal moves. The graph structure allows the implementation of search algorithms including Depth-First Search (DFS), Breadth-First Search (BFS) and the Minimax algorithm with Alpha-Beta pruning. These algorithms serve artificial intelligence systems for decision making under uncertain conditions with restricted resources which makes them suitable for chess board evaluation. The Knight's Tour problem serves as a Hamiltonian path example in graph theory to show graph theory usage.

The main goal of this report project is to demonstrate how graph-based approaches together with mathematical logic enable the functionality of human-level decision-making in the complex chess game. This report uses a self-developed chess engine to show how discrete mathematical concepts lead to practical applications which then shows the relationships between game theory and graph algorithms and AI programming.

Some parts of this report include open-source code and theory which are cited and used solely for academic purposes.

---

2. Mathematical Concepts

This section explains the key discrete mathematics principles used in this project. Each subsection covers an algorithm or structure and how it applies to my chess engine and graph-based modeling.

2.1 Graph Theory in Chess

The principles of graph theory enable the creation of chess models together with piece movement rules. In this context:

- The squares on the board are the vertices
- The board will then change after every move

A knight starting from square d4 can move to any of its 8 legal possible boxes through a subgraph representing legal moves. This representation helps chess engines to use graphs and search algorithms for gameplay simulations.

2.2 Game Trees and Decision Making

A game tree functions as a branching system which displays board states through nodes while legal moves connect these nodes as edges. The chess engine uses many branches to determine upcoming potential outcomes.

- The current board state functions as the root node of the structure.
- The child nodes contain all possible outcomes that result from legal moves.

The engine performs alternating operations at each level between:

- Maximizing player (AI)
- Minimizing player (opponent)

The decision-making process under uncertainty becomes possible through this structure because it allows the evaluation and comparison of future outcomes across many moves.

2.3 Search Algorithms: DFS and BFS

Depth-First Search (DFS)

The DFS algorithm explores one game tree branch to its deepest extent before it starts backtracking. The recursive Minimax algorithm employs DFS to simulate numerous sequential future moves. (GeeksforGeeks, n.d.)

The memory efficiency of DFS makes it the fundamental traversal method in most chess engines because it handles complex decision trees effectively.

Breadth-First Search (BFS)

BFS explores every possible move at the current level before it advances to deeper levels. BFS serves specific purposes although it does not frequently appear in deep chess decision-making processes. (Programiz, n.d.; GeeksforGeeks, n.d.)

- Finding shortest paths
- Solving puzzles like Knight's Tour

The game decision tree becomes accessible through both algorithms which use graph principles for traversal.

2.4 Minimax Algorithm and Alpha-Beta Pruning

The Minimax algorithm determines the best move through by simulating of all possible actions between players.

- The Maximizer (AI) selects moves which produce the most score.
- The Minimizer (opponent) aims to reduce that score.

The algorithm performs a repeated exploration of all the possible moves and the countermoves until it would reach a predetermined depth limit before evaluating the result. (GeeksforGeeks, n.d.)

Alpha-Beta Pruning

The optimization technique of Minimax removes unnecessary branches which don't affect the final move.

- The best score that the maximizer can get remains at Alpha level.
- The best score that the minimizer can get remains at Beta level.

The algorithm cuts off branches when their score falls below the existing evaluated scores. The engine needs to evaluate fewer times because of this optimization. (GeeksforGeeks, n.d.)

2.5 The Knight's Tour Problem

The Knight's Tour shows a classic Hamiltonian path problem which requires visiting each square on the board exactly once through knight movements. (Wikipedia contributors, 2024)

From a graph theory perspective:

- Each square on the chess board is a node
- Each legal knight move is an edge (2 blocks in any direction and then either left or right)
- The goal is to find a path that visits every node once

The Knight's Tour serves as an example to demonstrate graph traversal algorithms. The problem presents both visual and mathematical depth which makes it perfect for illustrating graph theory through game-based examples. (Wikipedia contributors, 2024)

3. Application: AI Chess Engine

This section shows how graph theory and search algorithms concepts are used in developing chess engine. The program uses the Minimax algorithm with alpha-beta pruning to evaluate moves during gameplay simulation. The chessboard is represented as a dynamic graph which the program uses traversal algorithms to explore possible outcomes. The engine uses the python-chess library together with logic to simulate AI behavior.

3.1 Move Evaluation and Scores

The engine uses a material-based evaluation function to assess the quality of each board state. This function assigns values to pieces and calculates a score based on the difference between White's and Black's material:

```
python
CopyEdit
PIECE_VALUES = {
    chess.PAWN: 1,
    chess.KNIGHT: 3,
    chess.BISHOP: 3,
    chess.ROOK: 5,
    chess.QUEEN: 9,
    chess.KING: 0
}
```

The logic is implemented and available in the source code (Mahmood, 2025).

The evaluation function:

- Rewards material advantage.
- Penalizes checkmate with ±9999.
- Returns 0 for draws).

The score helps the engine to assign weights to graph nodes when constructing and evaluating the game tree.

3.2 Implementing Graph-Based Search

The chess engine uses the **Minimax algorithm** with **alpha-beta pruning** to select the best move:

```
python
CopyEdit
def get_best_move(board, depth):
    best_move = None
    best_value = -float('inf')
    for move in board.legal_moves:
        board.push(move)
        move_value = minimax(board, depth - 1, -float('inf'), float('inf'),
False)
        board.pop()
        if move_value > best_value:
            best_value = move_value
            best_move = move
    return best_move
```

The logic is implemented and available in the source code (Mahmood, 2025).

The search method performs DFS on the game tree until it reaches a predetermined depth. Alpha-beta pruning enhances efficiency by eliminating least scored options.

The algorithm creates directed edges between legal moves to form new board states (vertices) until it reaches either the maximum search depth.

3.3 Visual Demonstrations

The graphical interface (chess_gui.py) allows users to play against the AI and observe its decision-making in real time. The engine prints board states and selected moves after each turn.

During execution:

- The user plays as White, entering moves via input.
- The AI responds using the best move found by Minimax.
- The printed board updates after each turn, providing a visual trail of moves and their consequences.

Although not visualized as a formal graph, the underlying logic treats each move as a traversal within a dynamically evolving graph.

The full implementation is available on GitHub (Mahmood, 2025).

4. Conclusion

The project illustrates how graph theory and search algorithms from discrete mathematics serve as the base for artificial intelligence applications in strategic games like chess. The chessboard graph structure and game tree representation helps us to use Minimax and Depth-First Search and Breadth-First Search and Alpha-Beta pruning algorithms for simulating intelligent game decisions. The chess engine shows these concepts by evaluating game states while predicting outcomes and selecting optimal moves through mathematical logic. The evaluation function uses weighted heuristics to create numerical assessments that guide the AI's strategic decisions. Through its dynamic game tree the engine helps both the evaluation of its own options and the assessment of potential responses from the opponent. The Knight's Tour functions as a theoretical and visual demonstration of how graph traversal problems can be studied within the framework of this well-known game. The classic puzzle shows how complex real-world choices can be transformed into structured mathematical problems. The project shows that graph theory combined with algorithmic reasoning creates both a method to understand games and a pathway toward sophisticated artificial intelligence and optimization techniques. Discrete mathematics functions as a powerful tool which helps machines to think.

5. References

Rosen, K. H. (2019). *Discrete mathematics and its applications* (8th ed.). McGraw-Hill Education.

Mahmood, M. T. (2025). *CHESS-AI-ENGINE* [Computer software]. GitHub. https://github.com/MuhammadTaahaMahmood-19/CHESS-AI-ENGINE

GeeksforGeeks. (n.d.). *Minimax algorithm in game theory*. Retrieved July 27, 2025, from https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/

GeeksforGeeks. (n.d.). *Alpha-beta pruning in game tree*. Retrieved July 27, 2025, from https://www.geeksforgeeks.org/alpha-beta-pruning-in-game-tree/

GeeksforGeeks. (n.d.). Depth First Search or DFS for a Graph. Retrieved July 27, 2025, from https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/

GeeksforGeeks. (n.d.). Breadth First Search or BFS for a Graph. Retrieved July 27, 2025, from https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/

Wikipedia contributors. (2024, May 10). *Knight's tour*. Wikipedia. https://en.wikipedia.org/wiki/Knight%27s_tour

Programiz. (n.d.). Breadth First Search (BFS) Algorithm. Retrieved July 27, 2025, from https://www.programiz.com/dsa/graph-bfs